COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/07/2026	**DATE SUBMITTED:**	10/09/2026
NAME:	Antonio, Christine Mae P. Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A:

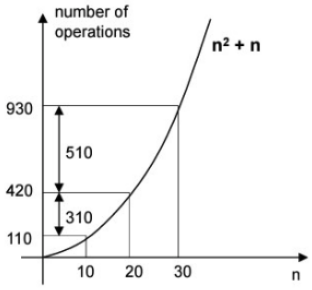
Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
        for(int i = 0; i < y.length; i++){
                if(search(x, y[i])!= - 1){
                        return false;
                }
        }
        return true;
}
```

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

- **Libo-on:** The graph shows that as n increases, the number of operations also increases by $n^2 + n$. The time complexity of the given indicates an $O(n^2)$ or a quadratic growth followed by n.

- **Antonio:** The graph shows the increase of number of operation by $n^2 + n$ with respect to n. Specifically, the

worst case is n = 30 with 930 numbers of operations. From n = 20, the numbers of operations increased by 510, showing a quadratic time growth $O(n^2)$.

ILO B:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0ms	Time taken to search = 0 microseconds Student (379 572) needs vaccination.	The input size 1000 runs on a 0ms execution speed
10000	0ms	Time taken to search = 0 microseconds Student (379 572) needs vaccination.	The input size 10000 runs on a 0ms execution speed
100000	1ms	Time taken to search = 1 microseconds Student (379 572) needs vaccination.	The input size 100000 runs on a 1ms execution speed

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0ms	Time taken to search = 0 microseconds Student (727 562) does not need vaccination.	The search ended in 0ms with n = 1000
10000	0ms	Time taken to search = 0 microseconds Student (727 562) needs vaccination.	The search ended in 0ms with n = 10000
100000	0ms	Time taken to search = 0 microseconds Student (727 562) needs vaccination.	The search ended in 0ms with n = 100000

Analysis:

Libo-on: The binary search algorithm runs on an O(log n) time complexity which means that the number of operations grows slowly in proportion to the size of the array. Which is why the execution time is almost the same for each input size.

```
Time taken to search = 0 microseconds
Student (379 572) needs vaccination.
Time taken to search = 0 microseconds
Student (379 572) needs vaccination.
Time taken to search = 1 microseconds
Student (379 572) needs vaccination.
```

Figure1. Output of ILO B

Antonio: The input size increased by 100 times from 1000 to 100000, yet the execution speed remained 0ms. It indicates an O(log n) time complexity.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Theoretical Analysis:

- **Antonio:** The algorithm checks every pair of elements in the array to determine whether any two elements are equal. The best time complexity will be O(1) if there is a duplicate in the first comparison, and the worst time complexity is $O(n^2)$ if all elements are unique or does ot exist.

- **Libo-on:** The best case time complexity will be O(1) if the first two identical integers are found in the front indices. The worst time complexity is $O(n^2)$ when all numbers in the array are unique and the identical numbers are found in the last indices.

Experimental Analysis:

Code Snippet
Libo-on

```cpp
#include <iostream>
#include <vector>
#include <unordered_set>
#include <chrono>

bool isUniqueSet(const std::vector<int>& arr) {
    std::unordered_set<int> seen;
    for (int num : arr) {
        if (seen.count(num) > 0) {
            return false;
        }
        seen.insert(num);
    }
    return true;
}

void time_complex(const std::vector<int>& arr) {
    auto begin = std::chrono::steady_clock::now();
    bool result = isUniqueSet(arr);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();

    std::cout << "Result: " << std::boolalpha << result
              << " | Time taken: " << duration << " nanoseconds\n";
}

int main() {
    time_complex({1, 2, 3, 4});
    time_complex({1, 2, 3, 1});
    return 0;
}
```

Antonio

```cpp
#include <iostream>
#include <vector>
#include <unordered_set>
#include <chrono>

bool isUniqueSet(const std::vector<int>& arr) {
    std::unordered_set<int> seen;

    for (int num: arr) {
        if (seen.count(num) > 0) {
            return false;
        }
        seen.insert(num);
    }
    return true;
}

void time_complex(const std::vector<int>& arr) {
    auto begin = std::chrono::steady_clock::now();
    bool result = isUniqueSet(arr);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();

    std::cout << "Result: " << std::boolalpha << result << "\nTime taken: " << duration << " nanoseconds\n";

}

int main() {
    time_complex({1, 2, 3});
    time_complex({1, 2, 1});

    return 0;
}
```

Figure2. Code Snippet of ILOA Supplementary

Output
Libo-on
Result: true | Time taken: 11200 nanoseconds Result: false | Time taken: 11300 nanoseconds
Antonio
Result: true Time taken: 8720 nanoseconds Result: false Time taken: 1870 nanoseconds

Figure2. Output of ILOA Supplementary

- **Antonio:** Two arrays were used with varying results. The first array [1, 2, 3] returned true because it has no duplicate. Its execution time took 8720 nanoseconds as it searched though the entire array. The second array [1, 2, 1] returned false because it is not unique. It took 1870 nanoseconds, returning result when it found the duplicate. This indicates a best time complexity of O(n) and worst time complexity of $O(n^2)$.

- **Libo-on:** The code snippet above shows the comparison between two arrays. One prints an unidentical; array, and the other prints an identical array. The first array [1,2,3,4] shows true because there are no identical values in it. While the second array [1,2,3,1] prints false because there are identical values. The execution time between the two arrays are almost the same at 11200-11300 nanoseconds at O(1) time complexity and $O(n^2)$ as its worst time complexity.

Analysis and Comparison:

- **Antonio:** The algorithm has a best time complexity of O(n) and worst time complexity of $O(n^2)$ as shown in the program, with the first array having less time than the second array.

- **Libo-on:** The theoretical analysis provided the initial assumption for the time complexity of the program at ideal levels. The experimental analysis proved that the theoretical analysis was true by providing detailed information and data to prove that the time complexity of the array is at O(1) at best and $O(n^2)$ at worst.

Problem 2

Theoretical Analysis:

- **Antonio:** The algorithm rpower calculates x^n by repeatedly multiplying x to x^{n-1} to get the power. The time complexity is O(n) as the recursive call reduces n by 1, therefore there are n + 1 calls. The algorithm brpower calculates x^n by dividing n to 2 at every recursive call. Its time complexity is O(log n), which means that the running time will increase by constant.

- **Libo-on:** The rpower uses linear recursion to get the value of n. This yields a time complexity of O(n) for linear representation. For each step, the linear representation decrements one to the exponent of x making it look like $x * x^{n-1}$ as the general formula. It has a tendency to trigger stack overflow because of the multiplying elements in a linear manner. On the other hand, the brpower gets the value of n by binary recursion which has a time complexity of O(log n). It halves the exponent each step making it $x * x^{n/2}$ as the general formula.

Experimental Analysis:

Code Snippet
Libo-on

```cpp
#include <iostream>
#include <chrono>

//RPower - O(n)
long long rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

// BRPower - O(Log n)
long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

void test_rpower(int x, int n) {
    auto begin = std::chrono::steady_clock::now();
    long long result = rpower(x, n);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
    std::cout << "rpower(" << x << ", " << n << ") | Time: " << duration
              << " nanoseconds | Result: " << result << std::endl;
}

void test_brpower(int x, int n) {
    auto begin = std::chrono::steady_clock::now();
    long long result = brpower(x, n);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
    std::cout << "brpower(" << x << ", " << n << ") | Time: " << duration
              << " nanoseconds | Result: " << result << std::endl;
}
```

Antonio

```cpp
#include <iostream>
#include <chrono>

long long rpower (int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n-1);
}

long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        long long y = brpower(x, (n-1) /2);
        return x * y * y;
    } else {
        long long y = brpower(x, n/2);
        return y*y;
    }
}

void test_rpower(int x, int n) {
    auto begin = std::chrono::steady_clock::now();
    long long result = rpower(x, n);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();

    std::cout << "rpower(" << x << ", " << n << "| Time: " << duration << " nanoseconds | Result: " << result << std::endl;
}

void test_brpower(int x, int n) {
    auto begin = std::chrono::steady_clock::now();
    long long result = brpower(x, n);
    auto end = std::chrono::steady_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
    std::cout << "brpower(" << x << ", " << n << "| Time: " << duration << " nanoseconds | Result: " << result << std::endl;

}

int main() {

    test_rpower(1, 1000);
    test_rpower(1, 100000);
    test_brpower(1, 1000);
    test_brpower(1, 100000);

    return 0;
}
```

Figure3. Code Snippet of ILOA Supplementary

Output
Libo-on
rpower(1, 1000) | Time: 30000 nanoseconds | Result: 1 rpower(1, 10000) | Time: 419100 nanoseconds | Result: 1 rpower(1, 5000) | Time: 34500 nanoseconds | Result: 1 brpower(1, 1000) | Time: 200 nanoseconds | Result: 1 brpower(1, 10000) | Time: 200 nanoseconds | Result: 1 brpower(1, 100000) | Time: 200 nanoseconds | Result: 1

Antonio

```
rpower(1, 1000| Time: 41611 nanoseconds | Result: 1
rpower(1, 100000| Time: 17868239 nanoseconds |
Result: 1
brpower(1, 1000| Time: 210 nanoseconds | Result: 1
brpower(1, 100000| Time: 210 nanoseconds | Result: 1
```

Figure3. Output of ILOA Supplementary

- **Antonio:** The running time for rpower increased by how much n increased. The time complexity is O(n). For the brpower, its runtime is at consistent 210 nanoseconds for all calls.

- **Libo-on:** The linear representation had an error occur when 100000 size is utilized. It triggered a stack overflow. It also had a significantly slower execution time as the size grew. On the other hand, the binary recursion had a consistent 200 nanosecond execution time even at higher scales of size.

Analysis and Comparison:

- **Libo-on:** The theoretical analysis closely aligns with the experimental analysis because the linear representation, rpower, uses linear recursion technique which is slower and scales depending on how many input sizes are in the array. Moreover, linear representation risks getting stack overflow for larger input sizes like 50000 or 100000. On the other hand, the binary recursion, brpower, achieved 200 nanoseconds regardless of how large the sample size is. Which also aligns with the theoretical analysis.

- **Antonio:** The experimental analysis shows that the theoretical analysis is true. The rpower algorithm's running time increased by double as n increased. As shown, n = 10000 resulted in 17868239 nanoseconds compared to n = 1000 with 41611 nanoseconds. The brpower running time stayed constant, reaching 0ms in all n.

Conclusion

Libo-on: In this activity, I learned how to use chrono library to get the actual time execution of the program as well as how the recursion method affects the time complexity and time execution. I also learned about how important the runtime is and how it affects the program execution especially when dealing with large samples.

Antonio: The activity demonstrated the relationship between the size of array and number of operations. When the time complexity is O(n), the runtime increases proportional to the array size.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.